Задания №19-21, программирование | Часть 3

Уровень: "Важный"

Задание №1

Павел и Виталий играют в одну игру.
Смысл игры заключается в следующем:
1) Перед игроками лежат две кучи камней. Игроки ходят по очереди, первый ход делает Павел.
2) За один ход игрок может добавить в одну из двух куч на своё усмотрение 1 камень или увеличить количество камней в куче в 3 раза. Для этого у каждого мальчика есть неограниченный запас камней.
3) Игра завершается тогда, когда количество камней в двух кучах стало не менее 66.
4) Победителем является тот игрок, который сделал последний ход и получил сумму камней в куче, равную 66 или больше.
Пример: имея кучи (10, 5) камней, за один ход игрок может получить: (11, 5), (10, 6), (30, 5), (10, 15).
В начальный момент в кучах было (6, N) камней, где N в диапазоне [1, 59].
Примечание. Игрок обладает выигрышной стратегией, если у него есть возможность выиграть, набрав нужную сумму камней в куче при любых ходах противника.
Мы знаем, что Виталий выиграл своим первым ходом после неудачного первого хода Павла. Определите минимальное начальное количество камней в куче, при котором возможна такая ситуация.

INSANEK.RU

Верные варианты

7

Решение

Сообщить об ошибке

Решение программированием. Способ 1:
1. Объявляем функцию g(s, p, end), которая будет вычислять результат игры, где s — текущее количество камней в куче, p — текущий ход игры, end — количество ходов, за которое должна завершиться игра.
2. Внутри функции пропишем, что если сумма камней в кучах не менее 66 и игразавершилась за необходимое количество ходов, то вернем истину, в остальных случаях —ложь.
3. Если следующий ход будет такой же четности, что и последний, то нам нужна хотя бы одним ходом, иначе — всеми вариантами ходов.
4. Запустим цикл для перебора значений S. Если при текущем значении игра может завершиться за два хода победой Виталия при неудачном ходе Павла, то выведем S.
Код:

```
def g(x, s, p, end):
   if (x + s) >= 66: return p in end
   if (x + s) < 66 and p == max(end): return False
   moves = [g(x + 1, s, p + 1, end), g(x * 3, s, p + 1, end), g(x, s + 1, p + 1, end), g(x, s * 3, p + 1, end)]
   return any(moves)
x = 6
print([s for s in range(1, 66) if g(x, s, 0, [2])])
```

Решение через функции all() и any():
1. Импортируем библиотеку functools для того, чтобы использовать декоратор lru_cache.
2. Напишем функцию, в которой мы распишем варианты всех ходов.
3. Запишем функцию f(h), перед которой мы запишем декоратор lru_cache. Сама функция будет возвращать результат игры.
4. Найдем все значения S, при которых Виталий выиграет первым ходом после неудачного хода Павла.

```
from functools import lru_cache

def moves(h):
   a, b = h
   return (a + 1, b), (a * 3, b), (a, b + 1), (a, b * 3)

@lru_cache(None)
def f(h):
   if sum(h) >= 66:
      return 'End'
   elif any(f(x) == 'End' for x in moves(h)):
      return 'P1'

task_19 = []
for i in range(1, 60):
   h = 6, i
   if any(f(x) == 'P1' for x in moves(h)):
      task_19.append(i)
print(min(task_19))
```

Задание №2

Павел и Виталий играют в одну игру.
Смысл игры заключается в следующем:
1) Перед игроками лежат две кучи камней. Игроки ходят по очереди, первый ход делает Павел.
2) За один ход игрок может добавить в одну из двух куч на своё усмотрение 1 камня или увеличить количество камней в куче в 3 раза. Для этого у каждого мальчика есть неограниченный запас камней.
3) Игра завершается тогда, когда количество камней в двух кучах стало не менее 66.
4) Победителем является тот игрок, который сделал последний ход и получил сумму камней в куче, равную 66 или больше.
Пример: имея кучи (10, 5) камней, за один ход игрок может получить: (11, 5), (10, 6), (30, 5), (10, 15).
В начальный момент в кучах было (6, N) камней, где N в диапазоне [1, 59].
Примечание. Игрок обладает выигрышной стратегией, если у него есть возможность выиграть, набрав нужную сумму камней в куче при любых ходах противника.
Найдите значение N, при котором у Павла есть выигрышная стратегия, при условии, что параллельно выполняются два условия:
— Павел не может выиграть за один ход;
— Павел может выиграть своим вторым ходом независимо от того, как будет ходить Виталий.

Верные варианты

11

INSANEK.RU

Решение

Сообщить об ошибке

Решение программированием. Способ 1:
1. Объявляем функцию g(s, p, end), которая будет вычислять результат игры, где s — текущее количество камней в куче, p — текущий ход игры, end — количество ходов, за которое должна завершиться игра.
2. Внутри функции пропишем, что если сумма камней в кучах не менее 66 и игразавершилась за необходимое количество ходов, то вернем истину, в остальных случаях —ложь.
3. Если следующий ход будет такой же четности, что и последний, то нам нужна хотя бы одним ходом, иначе — всеми вариантами ходов.
4. Запустим цикл для перебора значений S. Если при текущем значении игра может завершиться за три хода, то выведем S.
Код:

```
def g(x, s, p, end):
  if (x + s) >= 66: return p in end
  if (x + s) < 66 and p == max(end): return False
  moves = [g(x + 1, s, p + 1, end), g(x * 3, s, p + 1, end), g(x, s + 1, p + 1, end), g(x, s * 3, p + 1, end)]
  return any(moves) if (p + 1) % 2 == (end[0] % 2) else all(moves)
x = 6
print([s for s in range(1, 66) if g(x, s, 0, [3])])
```

Решение с помощью программы на языке Python.
1. Создаём функцию, которая будет проверять, является ли текущая позиция выигрышной за один ход. Назовём её Win1, а в качестве параметра будем передавать количество камней в первой куче и возможное начальное значение S.
Функция будет возвращать результат выражения (x + S < 66) and ((x + S + 1 >= 66) or (x * 3 + S >= 66) or (S * 3 + x >= 66)). То есть результатом будет True (если позиция подходит) или False (если позиция не подходит). Это помогает узнать, можно ли за один ход получить победное (>=66) количество камней в двух кучах:

```
def Win1(x, S):
  return (x + S < 66) and ((x + S + 1 >= 66) or (x * 3 + S >= 66) or (S * 3 + x >= 66))
```

2. Создаём функцию, которая будет проверять, является ли текущая позиция проигрышной за 1 ход. Проигрышная за 1 ход позиция — это позиция, из которой соперник 100% выиграет, как бы мы ни сходили (причём выиграет своим первым ходом).
Функция будет возвращать результат выражения ((not(Win1(x, S))) and Win1(x + 1, S) and Win1(x, S + 1) and Win1(x * 3, S) and Win1(x, S * 3)).
Что должно выполняться, чтобы функция вернула True?
1) Текущая позиция не должна быть выигрышной за 1 ход (not(Win1(x, S))).
2) Как бы ни сходил игрок, у него должна получаться такая позиция, из которой соперник победит своим первым ходом.

```
def Loss1(x, S):
  return ((not(Win1(x, S))) and Win1(x + 1, S) and Win1(x, S + 1) and Win1(x * 3, S) and Win1(x, S * 3))
```

3. Создаём функцию, которая будет проверять, является ли текущая позиция выигрышной за два хода. То есть может ли игрок, который будет из неё ходить, победить своим вторым ходом.
Функция будет возвращать результат выражения (Loss1(x + 1, S) or Loss1(x, S + 1) or Loss1(x * 3, S) or Loss1(x, S * 3)). Что должно выполняться, чтобы функция вернула True?
У игрока, который делает ход из этой позиции, должен быть хотя бы один вариант, который сможет принести проигрыш сопернику. То есть один из возможных ходов должен создать проигрышную за 1 ход позицию (поэтому пишем or), чтобы соперник не смог победить, а сам игрок своим вторым ходом одержал победу.

```
def Win2(x, S):
  return (Loss1(x + 1, S) or Loss1(x, S + 1) or Loss1(x * 3, S) or Loss1(x, S * 3))
```

4. Создаём список, в котором будут храниться все подходящие под условие значения S, и переменную, в которой будет храниться количество камней в первой куче:

```
x = 6
S20 = []
```

5. Создаём цикл для перебора возможных значений S. Диапазон перебора указан в условии задачи:

```
for S in range(1, 60):
```

6. Делаем проверку:

if Win2(x, S):
Она работает так: если из текущей позиции можно победить вторым ходом, то такая позиция подходит и мы добавим её в список:
S20.append(S)
7. Выводим на экран весь список и выбираем ответ:
print(S20)
Полный код:

```
def Win1(x, S):
  return (x + S < 66) and ((x + S + 1 >= 66) or (x * 3 + S >= 66) or (S * 3 + x >= 66))
def Loss1(x, S):
  return ((not(Win1(x, S))) and Win1(x + 1, S) and Win1(x, S + 1) and Win1(x * 3, S) and Win1(x, S * 3))
def Win2(x, S):
  return (Loss1(x + 1, S) or Loss1(x, S + 1) or Loss1(x * 3, S) or Loss1(x, S * 3))

x = 6
S20 = []
for S in range(1, 60):
  if Win2(x, S):
    S20.append(S)
print(S20)
```

INSANEK.RU

Решение через функции all() и any():
1. Импортируем библиотеку functools для того, чтобы использовать декоратор lru_cache.
2. Напишем функцию, в которой мы распишем варианты всех ходов.
3. Запишем функцию f(h), перед которой мы запишем декоратор lru_cache. Сама функция будет возвращать результат игры.
4. Найдем все значения S, при которых Павел выиграет вторым ходом.

```
from functools import lru_cache

def moves(h):
  a, b = h
  return (a + 1, b), (a * 3, b), (a, b + 1), (a, b * 3)

@lru_cache(None)
def f(h):
  if sum(h) >= 66:
    return 'End'
  elif any(f(x) == 'End' for x in moves(h)):
    return 'P1'
  elif all(f(x) == 'P1' for x in moves(h)):
    return 'V1'
  elif any(f(x) == 'V1' for x in moves(h)):
    return 'P2'

task_20 = []
for i in range(1, 60):
  h = 6, i
  if f(h) == 'P2':
    task_20.append(i)
print(task_20)
```

Задание №3

Павел и Виталий играют в одну игру.
Смысл игры заключается в следующем:
1) Перед игроками лежат две кучи камней. Игроки ходят по очереди, первый ход делает Павел.
2) За один ход игрок может добавить в одну из двух куч на своё усмотрение 1 камня или увеличить количество камней в куче в 3 раза. Для этого у каждого мальчика есть неограниченный запас камней.
3) Игра завершается тогда, когда количество камней в двух кучах стало не менее 66.
4) Победителем является тот игрок, который сделал последний ход и получил сумму камней в куче, равную 66 или больше.
Пример: имея кучи (10, 5) камней, за один ход игрок может получить: (11, 5), (10, 6), (30, 5), (10, 15).
В начальный момент в кучах было (6, N) камней, где N в диапазоне [1, 59].
Примечание. Игрок обладает выигрышной стратегией, если у него есть возможность выиграть, набрав нужную сумму камней в куче при любых ходах противника.
Найдите значение N, при котором параллельно выполняются два условия:
— у Виталия есть стратегия, позволяющая ему выиграть первым или вторым ходом при любой игре Павла;
— у Виталия нет стратегии, которая позволит ему гарантированно выиграть первым ходом.

Верные варианты

19

Решение

Сообщить об ошибке

Решение программированием. Способ 1:
1. Объявляем функцию g(s, p, end), которая будет вычислять результат игры, где s — текущее количество камней в куче, p — текущий ход игры, end — количество ходов, за которое должна завершиться игра.
2. Внутри функции пропишем, что если сумма камней в кучах не менее 66 и игразавершилась за необходимое количество ходов, то вернем истину, в остальных случаях —ложь.
3. Если следующий ход будет такой же четности, что и последний, то нам нужна хотя бы одним ходом, иначе — всеми вариантами ходов.
4. Запустим цикл для перебора значений S. Если при текущем значении игра может завершиться за
за 2 или 4 хода, то выведем S.
Код:

```
def g(x, s, p, end):
    if (x + s) >= 66: return p in end
    if (x + s) < 66 and p == max(end): return False
    moves = [g(x + 1, s, p + 1, end), g(x * 3, s, p + 1, end), g(x, s + 1, p + 1, end), g(x, s * 3, p + 1, end)]
    return any(moves) if (p + 1) % 2 == (end[0] % 2) else all(moves)
x = 6
print([s for s in range(1, 66) if g(x, s, 0, [2, 4]) and not(g(x, s, 0, [2]))])
```

INSANEK.RU

Решение через функции all() и any():
1. Импортируем библиотеку functools для того, чтобы использовать декоратор lru_cache.
2. Напишем функцию, в которой мы распишем варианты всех ходов.
3. Запишем функцию f(h), перед которой мы запишем декоратор lru_cache. Сама функция будет возвращать результат игры.
4. Найдем все значения S, при которых Павел выиграет вторым ходом.

```
from functools import lru_cache

def moves(h):
    a, b = h
    return (a + 1, b), (a * 3, b), (a, b + 1), (a, b * 3)

@lru_cache(None)
def f(h):
    if sum(h) >= 66:
        return 'End'
    elif any(f(x) == 'End' for x in moves(h)):
        return 'P1'
    elif all(f(x) == 'P1' for x in moves(h)):
        return 'V1'
    elif any(f(x) == 'V1' for x in moves(h)):
        return 'P2'
    elif all((f(x) == 'P2' or f(x) == 'P1') for x in moves(h)):
        return 'V2'

task_21 = []
for i in range(1, 60):
    h = 6, i
    if f(h) == 'V2':
        task_21.append(i)
print(task_21)
```

Уровень: "Нет данных"

Задание №4

Полина и Вероника играют в игру, правила которой заключаются в следующем:
1) Перед игроками лежат две кучи камней. Игроки ходят по очереди, первый ход делает Полина.
2) За один ход игрок может добавить в одну кучу 1 камень или увеличить количество камней в куче в 3 раза. Для этого у каждого игрока есть неограниченный запас камней.
3) Игра завершается в тот момент, когда сумма камней двух куч стала не менее 80.
4) Победителем является тот игрок, который сделал последний ход и получил сумму, равную 80 или больше.
Пример: из набора (5, 10) за один ход можно получить кучи камней: (6, 10), (15, 10), (5, 11), (5, 30).
В начальный момент в кучах было (3, S) камней, где S в диапазоне [1, 76].
Примечание. Игрок обладает выигрышной стратегией, если у него есть возможность набрать нужную сумму камней в кучах для выигрыша при любых ходах противника.
Мы знаем, что Вероника выиграла своим первым ходом после первого неудачного хода Полины. Определите минимальное начальное количество камней в куче, при котором возможна такая ситуация.

Верные варианты

9

Решение

Сообщить об ошибке

Решение программированием. Способ 1:
1. Объявляем функцию g(s, p, end), которая будет вычислять результат игры, где s — текущее количество камней в куче, p — текущий ход игры, end — количество ходов, за которое должна завершиться игра.
2. Внутри функции пропишем, что если сумма камней в кучах не менее 80 и игразавершилась за необходимое количество ходов, то вернем истину, в остальных случаях —ложь.
3. Если следующий ход будет такой же четности, что и последний, то нам нужна хотя бы одним ходом, иначе — всеми вариантами ходов.
4. Запустим цикл для перебора значений S. Если при текущем значении игра может завершиться за два хода победой Вани при неудачном ходе Пети, то выведем S.
Код:

INSANEK.RU

```
def g(x, s, p, end):
   if (x + s) >= 80: return p in end
   if (x + s) < 80 and p == max(end): return False
   moves = [g(x + 1, s, p + 1, end), g(x * 3, s, p + 1, end), g(x, s + 1, p + 1, end), g(x, s * 3, p + 1, end)]
   return any(moves)

x = 3
print([s for s in range(1, 77) if g(x, s, 0, [2])])
```

Решение программированием. Способ 2:
1. Создаём функцию, которая будет проверять, является ли текущая позиция выигрышной за один ход. Назовём её Win1, а в качестве параметра будем передавать количество камней в первой куче и возможное начальное значение S.
Функция будет возвращать результат выражения (x + S < 80) and (x + 1 + S >= 80 or (x * 3 + S >= 80) or (S * 3 + x >= 80)). То есть результатом будет True (если позиция подходит) или False (если позиция не подходит). Это помогает узнать, можно ли за один ход получить победное (>=80) количество камней в двух кучах:

```
def Win1(x, S):
   return (x + S < 80) and (x + 1 + S >= 80 or (x * 3 + S >= 80) or (S * 3 + x >= 80))
```

2. Создаём список, в котором будут храниться все подходящие под условие значения S, и переменную, в которой будет храниться количество камней в первой куче:

```
x = 3
S19 = []
```

3. Создаём цикл для перебора возможных значений S. Диапазон перебора указан в условии задачи:

```
for S in range(1, 77):
```

4. Делаем проверку:

```
if Win1(x + 1, S) or Win1(x, S + 1) or Win1(x * 3, S) or Win1(x, S * 3):
```

Она работает так: если после любого из ходов Полины (нас устроит хотя бы один подходящий вариант, поэтому пишем or) можно будет победить за 1 ход, то такая позиция подходит и мы добавим её в список:

```
S19.append(S)
```

5. Выводим на экран минимальное значение из списка — это и будет ответом:

```
print(min(S19))
```

Полный код:

```
def Win1(x, S):
   return (x + S < 80) and (x + 1 + S >= 80 or (x * 3 + S >= 80) or (S * 3 + x >= 80))
x = 3
S19 = []
for S in range(1, 77):
   if Win1(x + 1, S) or Win1(x, S + 1) or Win1(x * 3, S) or Win1(x, S * 3):
      S19.append(S)
print(min(S19))
```

Задание №5

Полина и Вероника играют в игру, правила которой заключаются в следующем:
1) Перед игроками лежат две кучи камней. Игроки ходят по очереди, первый ход делает Полина.
2) За один ход игрок может добавить в одну кучу 1 камень или увеличить количество камней в куче в 3 раза. Для этого у каждого игрока есть неограниченный запас камней.
3) Игра завершается в тот момент, когда сумма камней двух куч стала не менее 80.
4) Победителем является тот игрок, который сделал последний ход и получил сумму, равную 80 или больше.
Пример: из набора (5, 10) за один ход можно получить кучи камней: (6, 10), (15, 10), (5, 11), (5, 30).
В начальный момент в кучах было (3, S) камней, где S в диапазоне [1, 76].
Примечание. Игрок обладает выигрышной стратегией, если у него есть возможность набрать нужную сумму камней в кучах для выигрыша при

любых ходах противника.
Найдите значение S, при котором у Полины есть выигрышная стратегия, учитывая выполнение двух условий:
– Полина не может выиграть за один ход;
– Полина может выиграть своим вторым ходом независимо от того, как будет ходить Вероника.

Верные варианты

25

Решение

Сообщить об ошибке

Решение программированием. Способ 1:
1. Объявляем функцию g(s, p, end), которая будет вычислять результат игры, где s — текущее количество камней в куче, p — текущий ход игры, end — количество ходов, за которое должна завершиться игра.
2. Внутри функции пропишем, что если сумма камней в кучах не менее 80 и игразавершилась за необходимое количество ходов, то вернем истину, в остальных случаях —ложь.
INSANEK.RU
3. Если следующий ход будет такой же четности, что и последний, то нам нужна хотя бы одним ходом, иначе — всеми вариантами ходов.
4. Запустим цикл для перебора значений S. Если при текущем значении игра может завершиться за три хода, то выведем S.
Код:
```
def g(x, s, p, end):
   if (x + s) >= 80: return p in end
   if (x + s) < 80 and p == max(end): return False
   moves = [g(x + 1, s, p + 1, end), g(x * 3, s, p + 1, end), g(x, s + 1, p + 1, end), g(x, s * 3, p + 1, end)]
   return any(moves) if (p + 1) % 2 == (end[0] % 2) else all(moves)

x = 3
print([s for s in range(1, 77) if g(x, s, 0, [3])])
```
Решение программированием. Способ 2:
1. Создаём функцию, которая будет проверять, является ли текущая позиция выигрышной за один ход. Назовём её Win1, а в качестве параметра будем передавать количество камней в первой куче и возможное начальное значение S.
Функция будет возвращать результат выражения (x + S < 80) and (x + 1 + S >= 80 or (x * 3 + S >= 80) or (S * 3 + x >= 80)). То есть результатом будет True (если позиция подходит) или False (если позиция не подходит). Это помогает узнать, можно ли за один ход получить победное (>=80) количество камней в двух кучах:
```
def Win1(x, S):
   return (x + S < 80) and (x + 1 + S >= 80 or (x * 3 + S >= 80) or (S * 3 + x >= 80))
```
2. Создаём функцию, которая будет проверять, является ли текущая позиция проигрышной за 1 ход. Проигрышная за 1 ход позиция — это позиция, из которой соперник 100% выиграет, как бы мы ни сходили (причём выиграет своим первым ходом).
Функция будет возвращать результат выражения ((not(Win1(x, S))) and Win1(x + 1, S) and Win1(x, S + 1) and Win1(x * 3, S) and Win1(x, S * 3)).
Что должно выполняться, чтобы функция вернула True?
1) Текущая позиция не должна быть выигрышной за 1 ход (not(Win1(x, S))).
2) Как бы ни сходил игрок, у него должна получаться такая позиция, из которой соперник победит своим первым ходом.
```
def Loss1(x, S):
   return ((not(Win1(x, S))) and Win1(x + 1, S) and Win1(x, S + 1) and Win1(x * 3, S) and Win1(x, S * 3))
```
3. Создаём функцию, которая будет проверять, является ли текущая позиция выигрышной за два хода. То есть может ли игрок, который будет из неё ходить, победить своим вторым ходом.
Функция будет возвращать результат выражения (Loss1(x + 1, S) or Loss1(x, S + 1) or Loss1(x * 3, S) or Loss1(x, S * 3)). Что должно выполняться, чтобы функция вернула True?
У игрока, который делает ход из этой позиции, должен быть хотя бы один вариант, который сможет принести проигрыш сопернику. То есть один из возможных ходов должен создать проигрышную за 1 ход позицию (поэтому пишем or), чтобы соперник не смог победить, а сам игрок своим вторым ходом одержал победу.
```
def Win2(x, S):
   return (Loss1(x + 1, S) or Loss1(x, S + 1) or Loss1(x * 3, S) or Loss1(x, S * 3))
```
4. Создаём список, в котором будут храниться все подходящие под условие значения S, и переменную, в которой будет храниться количество камней в первой куче:
```
x = 3
S20 = []
```
5. Создаём цикл для перебора возможных значений S. Диапазон перебора указан в условии задачи:
```
for S in range(1, 77):
```
6. Делаем проверку:
```
if Win2(x, S):
```
Она работает так: если из текущей позиции можно победить вторым ходом, то такая позиция подходит и мы добавим её в список:
```
S20.append(S)
```
7. Выводим на экран весь список и выбираем ответ:
```
print(S20)
```

Полный код:
```
def Win1(x, S):
   return (x + S < 80) and (x + 1 + S >= 80 or (x * 3 + S >= 80) or (S * 3 + x >= 80))
def Loss1(x, S):
   return ((not(Win1(x, S))) and Win1(x + 1, S) and Win1(x, S + 1) and Win1(x * 3, S) and Win1(x, S * 3))
```

```
def Win2(x, S):
   return (Loss1(x + 1, S) or Loss1(x, S + 1) or Loss1(x * 3, S) or Loss1(x, S * 3))
x = 3
S20 = []
for S in range(1, 77):
   if Win2(x, S):
      S20.append(S)
print(S20)
```

Задание №6

Полина и Вероника играют в игру, правила которой заключаются в следующем:
1) Перед игроками лежат две кучи камней. Игроки ходят по очереди, первый ход делает Полина.
2) За один ход игрок может добавить в одну кучу 1 камень или увеличить количество камней в куче в 3 раза. Для этого у каждого игрока есть неограниченный запас камней.
3) Игра завершается в тот момент, когда сумма камней двух куч стала не менее 80.
4) Победителем является тот игрок, который сделал последний ход и получил сумму, равную 80 или больше.
Пример: из набора (5, 10) за один ход можно получить кучи камней: (6, 10), (15, 10), (5, 11), (5, 30).
В начальный момент в кучах было (3, S) камней, где S в диапазоне [1, 76].
Примечание. Игрок обладает выигрышной стратегией, если у него есть возможность набрать нужную сумму камней в кучах для выигрыша при любых ходах противника.
Найдите минимальное значение S, при котором параллельно выполняются два условия:
– у Вероники есть стратегия, позволяющая ей выиграть первым или вторым ходом при любой игре Полины;
– у Вероники нет стратегии, которая позволит ей гарантированно выиграть первым ходом.

INSANEK.RU

Верные варианты

24

Решение

Сообщить об ошибке

Решение программированием. Способ 1:
1. Объявляем функцию g(s, p, end), которая будет вычислять результат игры, где s — текущее количество камней в куче, p — текущий ход игры, end — количество ходов, за которое должна завершиться игра.
2. Внутри функции пропишем, что если сумма камней в кучах не менее 80 и игразавершилась за необходимое количество ходов, то вернем истину, в остальных случаях —ложь.
3. Если следующий ход будет такой же четности, что и последний, то нам нужна хотя бы одним ходом, иначе — всеми вариантами ходов.
4. Запустим цикл для перебора значений S. Если при текущем значении игра может завершиться за два или за четыре хода, то выведем S.
Код:

```
def g(x, s, p, end):
   if (x + s) >= 80: return p in end
   if (x + s) < 80 and p == max(end): return False
   moves = [g(x + 1, s, p + 1, end), g(x * 3, s, p + 1, end), g(x, s + 1, p + 1, end), g(x, s * 3, p + 1, end)]
   return any(moves) if (p + 1) % 2 == (end[0] % 2) else all(moves)

x = 3
print([s for s in range(1, 77) if g(x, s, 0, [2, 4])])
```

Решение программированием. Способ 2:
1. Создаём функцию, которая будет проверять, является ли текущая позиция выигрышной за один ход. Назовём её Win1, а в качестве параметра будем передавать количество камней в первой куче и возможное начальное значение S.
Функция будет возвращать результат выражения (x + S < 80) and (x + 1 + S >= 80 or (x * 3 + S >= 80) or (S * 3 + x >= 80)). То есть результатом будет True (если позиция подходит) или False (если позиция не подходит). Это помогает узнать, можно ли за один ход получить победное (>=80) количество камней в двух кучах:

```
def Win1(x, S):
   return (x + S < 80) and (x + 1 + S >= 80 or (x * 3 + S >= 80) or (S * 3 + x >= 80))
```

2. Создаём функцию, которая будет проверять, является ли текущая позиция проигрышной за 1 ход. Проигрышная за 1 ход позиция — это позиция, из которой соперник 100% выиграет, как бы мы ни сходили (причём выиграет своим первым ходом).
Функция будет возвращать результат выражения ((not(Win1(x, S))) and Win1(x + 1, S) and Win1(x, S + 1) and Win1(x * 3, S) and Win1(x, S * 3)).
Что должно выполняться, чтобы функция вернула True?
1) Текущая позиция не должна быть выигрышной за 1 ход (not(Win1(x, S))).
2) Как бы ни сходил игрок, у него должна получаться такая позиция, из которой соперник победит своим первым ходом.

```
def Loss1(x, S):
   return ((not(Win1(x, S))) and Win1(x + 1, S) and Win1(x, S + 1) and Win1(x * 3, S) and Win1(x, S * 3))
```

3. Создаём функцию, которая будет проверять, является ли текущая позиция выигрышной за два хода. То есть может ли игрок, который будет из неё ходить, победить своим вторым ходом.
Функция будет возвращать результат выражения (Loss1(x + 1, S) or Loss1(x, S + 1) or Loss1(x * 3, S) or Loss1(x, S * 3)). Что должно выполняться, чтобы функция вернула True?

У игрока, который делает ход из этой позиции, должен быть хотя бы один вариант, который сможет принести проигрыш сопернику. То есть один из возможных ходов должен создать проигрышную за 1 ход позицию (поэтому пишем or), чтобы соперник не смог победить, а сам игрок своим вторым ходом одержал победу.

def Win2(x, S):

return (Loss1(x + 1, S) or Loss1(x, S + 1) or Loss1(x * 3, S) or Loss1(x, S * 3))

4. Создаём функцию, которая будет проверять, является ли текущая позиция проигрышной за 1 или 2 хода (Вероника выигрывает за 1 или 2 хода, следовательно, Полина проигрывает за 1 или 2 хода).

Функция будет возвращать True или False в зависимости от значения выражения. Истина возможна при выполнении условия: позиции, которые можно получить из текущей, 100% выигрышные — либо за 1 ход, либо за 2.

Чтобы исключить вариант, что все позиции, которые можно получить из (x, S), выигрышные только за 1 ход, добавим условие (Win2(x + 1, S) or Win2(x, S + 1) or Win2(x * 3, S) or Win2(x, S * 3)). Оно обеспечит хотя бы один вариант победы за 2 хода.

def Loss12(x, S):

return ((Win1(x + 1, S) or Win2(x + 1, S)) and (Win1(x, S + 1) or Win2(x, S + 1)) and (Win1(x * 3, S) or Win2(x * 3, S)) and (Win1(x, S * 3) or Win2(x, S * 3)) and (Win2(x + 1, S) or Win2(x, S + 1) or Win2(x * 3, S) or Win2(x, S * 3)))

5. Создаём список, в котором будут храниться все подходящие под условие значения S, и переменную, в которой будет храниться количество камней в первой куче:

x = 3

S21 = []

INSANEK.RU

6. Создаём цикл для перебора возможных значений S. Диапазон перебора указан в условии задачи:

for S in range(1, 77):

7. Делаем проверку:

if Loss12(x, S):

Она работает так: если из текущей позиции можно проиграть первым или вторым ходом, то такая позиция подходит и мы добавим её в список:

S21.append(S)

8. Выводим на экран минимальное значение списка:

print(min(S21))

Полный код:

```
def Win1(x, S):
   return (x + S < 80) and (x + 1 + S >= 80 or (x * 3 + S >= 80) or (S * 3 + x >= 80))
def Loss1(x, S):
   return ((not(Win1(x, S))) and Win1(x + 1, S) and Win1(x, S + 1) and Win1(x * 3, S) and Win1(x, S * 3))
def Win2(x, S):
   return (Loss1(x + 1, S) or Loss1(x, S + 1) or Loss1(x * 3, S) or Loss1(x, S * 3))
def Loss12(x, S):
   return ((Win1(x + 1, S) or Win2(x + 1, S)) and (Win1(x, S + 1) or Win2(x, S + 1)) and (Win1(x * 3, S) or Win2(x * 3, S)) and (Win1(x, S * 3) or Win2(x, S *
3)) and (Win2(x + 1, S) or Win2(x, S + 1) or Win2(x * 3, S) or Win2(x, S * 3)))
x = 3
S21 = []
for S in range(1, 77):
   if Loss12(x, S):
      S21.append(S)
print(min(S21))
```